

Chapter 1

MCU Memory Architecture & Clocking

ENEL712 Embedded Systems Design — Lecture 3

Summary

Detailed look at AVR memory architecture (Flash, SRAM, EEPROM), the SRAM memory map and stack, the bus system, and clock-source selection (with the UART accuracy constraint).

Topics Covered

- Architectural fundamentals (Harvard vs Von Neumann)
- Microcontroller memory types
- SRAM usage, memory map, and the stack
- Bus system
- System clock architecture & UART accuracy constraint

1.1 Architectural Fundamentals

- Harvard: physically separate program and data memory and buses, enabling concurrent access and pipelined fetch-execute.
- Constants must be placed in program memory using PROGMEM to avoid wasting SRAM.
- Von Neumann: shared memory and bus; introduces the Von Neumann bottleneck where the CPU must wait for memory.

1.2 Microcontroller Memory Types

Memory	Size (AT90USB128)	Purpose	Key characteristics
Flash	128 KB (64K × 16-bit)	Non-volatile program code and constants	≥100,000 erase/write cycles; use <code>PROGMEM</code> and <code>pgm_read_byte()</code> to save SRAM
SRAM	8 KB (0x0100–0x20FF)	Run-time variables, buffers, stack	Volatile; no refresh needed; preceded by GPRs (0x00–0x1F) and I/O regs (0x20–0xFF)
EEPROM	4 KB	Persistent settings & calibration	Byte-addressable; ~3.4 ms/byte write; ~100,000 cycles/byte; avoid frequent or ISR writes; use wear leveling

1.3 SRAM Usage, Memory Map, and the Stack

- Variables grow upward from 0x0100; the stack grows downward from 0x20FF.
- Stack: LIFO buffer for local variables and return addresses; the 16-bit Stack Pointer decrements on push, increments on pop.
- Avoid the C heap (`malloc`): leads to fragmentation, non-deterministic timing, and stack-overflow risk.

1.4 Bus System

Bus	Carries
Address bus	Memory address for instruction or data access
Data bus	Data (8-bit on AT90 series) between ALU, memory, and peripherals
Control bus	Read, write, and enable lines that control data flow

1.5 System Clock Architecture

Clock Source	Accuracy	External components	compo-	Use case
Low-Power Crystal Oscillator	≈ ±20–50 ppm	Crystal + 2 capacitors		Required for UART, USB, accurate timing/PWM (<2 % error)
Calibrated Internal RC	≈ ±10 %	None		Quick prototyping with no accurate timing/comms needs
Low-Frequency Crystal	≈ ±5–20 ppm	Tiny crystal		RTC and ultra-low power wakeup

1.5.1 Clock Accuracy: The UART Constraint

UART is sensitive because TX and RX have separate clocks and must agree on a baud rate. Internal RC oscillator drift ($\pm 10\%$) causes the receiver to sample data bits incorrectly after just a few bits — accurate UART requires a crystal so baud-rate error stays below $\approx 2\%$.

1.6 Use Cases & Applications

- Choosing a clock source: USB and UART force a crystal because they need $< 2\%$ timing error; idle prototyping can use the internal RC.
- Low-power design: the low-frequency 32.768 kHz crystal lets the MCU keep an RTC running on tens of nA in deep sleep.
- Calibration data storage: EEPROM holds factory trims (e.g., ADC offset, sensor gain) that survive reflashing.
- Stack analysis: knowing SRAM ends at 0x20FF lets you place a canary value below the maximum observed stack and detect overflow.

1.7 Formulas & Calculations

1.7.1 UART baud-rate accuracy budget

A UART frame is 10 bits (1 start + 8 data + 1 stop). The receiver samples each bit near its centre. After the start bit, accumulated timing error must stay below half a bit period at the last bit.

- Allowed error per bit $\approx 0.5 / 9 \approx 5.5\%$.
- Combined transmitter + receiver tolerance must fit inside this — so each side is typically held to $\leq 2\%$.
- An internal RC at $\pm 10\%$ drifts by far more than the budget; a crystal at ± 50 ppm is essentially perfect.

1.7.2 EEPROM lifetime with wear leveling

Endurance L cycles per cell, write rate W writes per second, N cells dedicated to the data $\rightarrow$ expected lifetime $\approx N \times L / W$ seconds.

1.8 Memory Map Cheat Sheet (AT90USB128)

Region	Address range	Notes
GPRs	0x0000–0x001F	32 × 8-bit registers visible as memory
I/O Registers	0x0020–0x00FF	SFRs accessed via IN/OUT or memory ops
Extended I/O	0x0100–0x01FF	Additional peripheral SFRs on bigger AVRs
SRAM (data)	0x0100–0x20FF	Globals grow up; stack grows down from 0x20FF
Flash (program)	0x0000–0x1FFFF	Word-addressed (16-bit), 64K words = 128 KB
EEPROM	0x0000–0x0FFF	Separate address space accessed via SFRs

Key Definitions

Harvard Architecture

Separate buses and memory spaces for program (Flash) and data (SRAM), allowing concurrent access.

Von Neumann Architecture

Single shared bus and memory for program and data; limited by the Von Neumann bottleneck.

Flash Memory

Non-volatile program storage with finite write endurance (~100k cycles).

SRAM

Volatile RAM (8 KB on AT90USB128) for runtime variables, buffers, and the stack.

EEPROM

Non-volatile, byte-addressable memory (4 KB) for persistent configuration.

Stack

LIFO SRAM region for return addresses and locals; grows downward from the top.

PROGMEM

AVR-GCC mechanism to force constants into Flash and save SRAM.

Wear Leveling

Strategy to rotate write addresses across EEPROM to avoid hitting endurance limits.

Stack Overflow

Downward-growing stack collides with upward-growing globals; corrupts data.

Von Neumann Bottleneck

Performance limit caused by a single shared memory bus.

Stack Canary

Sentinel byte placed below the deepest expected stack point; corruption signals stack overflow.

Crystal PPM

Parts-per-million frequency tolerance; 50 ppm = 0.005 %.

Heap Fragmentation

Free memory broken into many small unusable holes after repeated malloc/free cycles.

Sources

- Lecture 3.pdf